Лекция + практика + ЛР11 Объект Image

Наиболее зрелищные эффекты при программировании на JavaScript достигаются при работе с графикой. Программирование графики в JavaScript опирается на объект Image, который характеризуется следующими свойствами, методами и событиями:

Свойства	Методы	События
Border	Нет	OnAbort
Complete		OnError
Height		OnLoad
Hspace		
Name		
Src		
Vspace		
Width		
Lowsrc		

Несмотря на такое обилие свойств, их абсолютное большинство можно только читать, но не изменять. Об этом свидетельствует, прежде всего, отсутствие методов. Но два свойства все, же можно изменять: **src и lowsrc.** Этого оказывается достаточно для множества эффектов с картинками.

Все объекты класса Image можно разделить на встроенные и порожденные программистом. Встроенные объекты – это картинки тегов IMG. Обращение к картинкам (**лекция 10).**

Пусть, например, в документе описано изображение

<img src="images/cat.jpg" id="cat_id" name="cat_name">

и оно является n-ым изображением, встречающимся в документе. К этому элементу img можно обратиться следующими способами:

1. **Как к элементу массива images, используя его индекс (индексация начинается с 0):**

window.document.images[n-1]

2. **Как к элементу массива images , используя значение атрибута name как ключ массива:**

window.document.images['cat_name']

3. **Используя значение атрибута name как свойство объекта:**

window.document.cat_name

4. **Используя значение атрибута id и свойство getElementById:**

window.document.getElementById('cat_id')

Свойства src и lowsrc определяют URL изображения, которое монтируется внутрь документа. При этом lowsrc определяет временное изображение, обычно маленькое, которое отображается, пока загружается основное изображение, чей URL указывается в атрибуте SRC тега IMG. Свойство src принимает значение атрибута SRC тега IMG. Программист может изменять значения и src, и lowsrc. Рассмотрим пример с src:

```
1  document.i2.src="images2.gif";
```

Как видно из этого примера, существует возможность модифицировать вмонтированную картинку за счет изменения значения свойства src встроенного объекта Image. Если вы в первый раз просматриваете данную страницу (т.е. картинки не закешированы браузером), то постепенное изменение картинки будет заметно.

Изменение картинки. Изменить картинку можно, только присвоив свойству src встроенного объекта Image новое значение. Пример

```
<DIV
  onMouseOver="document.rtwG.src='pic2/image2.jpg'"
  onMouseOut="document.rtwG.src='pic2/image1.jpg'">
<img  src="pic2/image2.jpg"  name="rtwG" >
</DIV>
```

Задание 1. - Сгенерировать код, позволяющий по наведению курсора мышки изменять одно изображение другим изображением.

Фрагмент кода показывает типовой прием замещения и восстановления картинки при проходе курсора мыши. Естественно, что менять можно не одну, а сразу несколько картинок.

Главное, тем не менее, не в том, что картинки замещаются, а в том, с какой скоростью они это делают. Для достижения нужного результата в начале страницы создаются массивы картинок, в которые перед отображением перекачивается, графика.

Пример 2. Меняется картинка по нажатию кнопки

```
<script language="JavaScript">
var flag = 0;
 function changeImage() {
   if (flag == 0) {
     document.img.src = 'pic2/image1.jpg';
     flag = 1;
   }
   else {
     document.img.src = 'pic2/image2.jpg';
     flag = 0;
   }
 }
</script>

<body>
<img name = 'img' src = 'pic2/image1.jpg' />
<form>
<input type = 'button' value = 'Сменить изображение' onClick = 'changeImage()' /> </form></body>
```

Задание 2. Сгенерировать код позволяющий, по нажатию на кнопку листать изображения.

Задание 2(1). Используя свойство visibility сгенерировать код, где по щелчку на первое изображение появляется второе изображение.

№4

Задание 3. Сгенерировать код позволяющий, по щелчку на ссылки выводить заданный текст в указанном месте веб – страницы.

На рис.1 показана загруженная страница имеющая 4 ссылки, при нажатии например, на Link2 в координатах top:100; left:250 выводиться Test2DIV2, при нажатии Link3 в том же месте страницы появляется Test3DIV3.

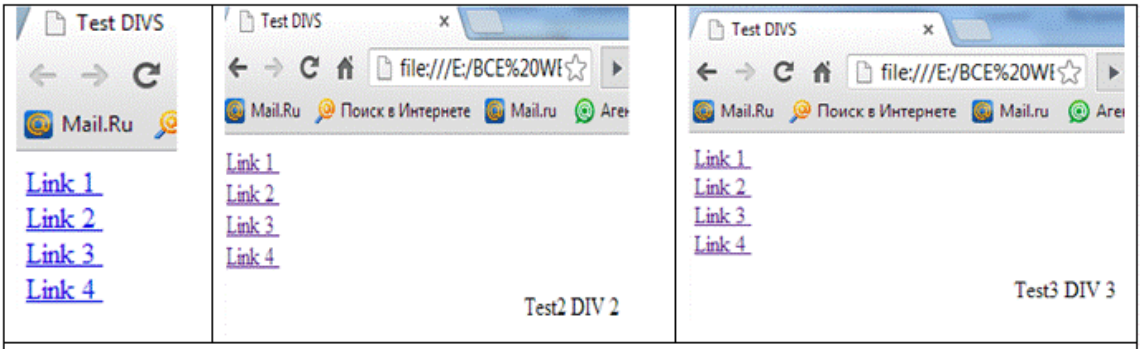


Рисунок 1

Естественным продолжением идеи замещения значения атрибута SRC в теге <IMG> является мультипликация, т.е. последовательное изменение значения этого атрибута во времени. Для реализации мультипликации используют **метод объекта Window – setTimeout()**.

Собственно, существуют два способа запуска мультипликации:

- **onLoad();**
- **onClick(), onChange() ...**

Наиболее популярный – setTimeout() при onLoad().

Событие onLoad() наступает в момент окончания загрузки документа браузером. Обработчик события указывается в теге <BODY>.

На первый взгляд, это просто рекурсия. Но в действительности все несколько сложнее. JavaScript разрабатывался для многопоточных операционных систем, поэтому правильнее будет представлять себе исполнение скриптов следующим образом:

1. Скрипт получает управление при событии onLoad().
2. Заменяет картинку.
3. Порождает новый скрипт и откладывает его исполнение на 1000 миллисекунд.
4. Текущий скрипт уничтожается JavaScript-интерпретатором.

После окончания срока задержки исполнения все повторяется. В данном примере (бесконечное повторение) функция порождает саму себя и тем самым поддерживает непрерывность своего выполнения.

Задание 4. Ориентируясь на выполненное задание 2-3 реализовать код в котором по щелчку на ссылки в области заданных координатах будет появляться изображения. Задать изображениям одинаковую высоту и ширину.

Задание 5. Сгенерировать код, в котором в начале страницы создается массив картинок, в который перед отображением перекачивается, графика.

5.1. Реализовать функцию, позволяющую осуществить последовательный вывод картинок. Задать в миллисекундах частоту мелькания кадров (каждую секунду). Для загрузки документа браузером использовать событие onLoad().

5.2. Реализовать функцию, позволяющую осуществить последовательный вывод картинок с заданным числом итераций. Задать в миллисекундах частоту мелькания кадров (каждую секунду). Для загрузки документа браузером использовать событие onLoad().

Перманентная мультипликация может быть достигнута и другими средствами, например многокадровыми графическими файлами. Однако движение на странице – не всегда благо. Часто возникает желание реализовать запуск и останов движения по требованию пользователя.

В данном случае просто осуществляется обход изменения картинки, но при этом не прекращаем порождение потока. Для решения проблемы остановки и старта мультипликации можно применить метод setTimeout() или метод clearTimeout().

Пример 3


```
<script language="JavaScript">
<!--
i=0;
img_a=new Array();   // Объявляется массив
img_a[0]=new Image();
img_a[1]=new Image();
img_a[2]=new Image();
img_a[3]=new Image();
img_a[4]=new Image();

img_a[0].src="pic2/image1.jpg"; // Каждому элементу
img_a[1].src="pic2/image2.jpg"; // массива присваивается
img_a[2].src="pic2/image3.jpg"; // конкретная картинка
img_a[3].src="pic2/image4.jpg";
img_a[4].src="pic2/image5.jpg";

function img_b()     // описывается функция последовательного
                     // вывода картинок
{document.images[0].src=img_a[i].src;
i++;
if(i>4) i=0;
setTimeout("img_b()", 1000); // Тут задается в миллисекундах // частота
мелькания кадров. В данном случае кадры меняются каждую секунду
}
//-->
</SCRIPT>
<!-- Скрипт закончен -->

</head>

<!-- В теге <body> помещаем вызов нашей функции --> <body
bgcolor="#ffffff" onLoad=img_b()>
<!-- А вот тут и будут листаться наши картинки -->
<img src="pic2/image4.jpg">
```

№7

Задание 6. Используя задание 4-5 реализовать запуск и остановку динамического показа изображений по требованию пользователя.

При программировании графики следует учитывать множество факторов, которые влияют на скорость отображения страницы и скорость изменения графических образов. При этом обычная дилемма оптимизации программ – скорость или размер занимаемой памяти – решается только путем увеличения скорости. О размере памяти при программировании на JavaScript думать как-то не принято.

Из всех способов оптимизации отображения картинок мы остановимся только на нескольких:

- оптимизация отображения при загрузке;
- оптимизация отображения за счет предварительной загрузки;
- оптимизация отображения за счет нарезки изображения.

Если первые две позиции относятся в равной степени, как к отображению статических картинок, так и к мультипликации, то третий пункт характерен главным образом для мультипликации.

Практически в любом руководстве по разработке HTML-страниц отмечается, что при использовании тега <IMG> в теле HTML-страницы следует указывать атрибуты WIDTH и HEIGHT. Это продиктовано порядком загрузки компонентов страницы с сервера и алгоритмом работы HTML-парсера. Первым загружается текст разметки. После этого парсер разбирает текст и начинает загрузку дополнительных компонентов, в том числе графики. При этом загрузка картинок, в зависимости от типа HTTP-протокола, может идти последовательно или параллельно.

Нарезка картинок применяется довольно часто. Она позволяет достигать эффекта частичного изменения отображаемой картинки. Чаще всего он применяется при создании меню.

Кроме подобного эффекта нарезка позволяет реализовать мультипликацию на больших картинках. При этом изменяется не весь образ, а только отдельные его части.

Одним из наиболее популярных приемов дизайна страниц Web-узла является техника нарезки картинок на составные части. Можно выделить следующие способы применения этой техники для организации навигационных компонентов страницы:

- горизонтальные и вертикальные меню;
- вложенные меню;
- навигационные графические блоки.

Задание 7. Самостоятельно разобраться и сгенерировать код вертикального и горизонтального меню, построенных на основе графических блоков текста.

Задание 8. Применив элемент canvas создать мультипликацию по своему усмотрению.

http://html5book.ru/html5/canvas

Также параллельно с загрузкой парсер продолжает свою работу. Если для картинок заданы параметры ширины и высоты, то можно отформатировать текст и отобразить его в окне браузера. До тех пор, пока эти параметры не определены, отображения текста не происходит.

Таким образом, указание высоты и ширины картинки позволит отобразить документ раньше, чем картинки будут получены с сервера. Это дает пользователю возможность читать документ или задействовать его гипертекстовые ссылки до момента полной загрузки (событие load).

С точки зрения JavaScript, указание размеров картинки задает начальные параметры окна отображения графики внутри документа. Это позволяет воспользоваться маленьким прозрачным образом для того, чтобы заменить его полноценной картинкой. Идея состоит в передаче маленького объекта для замещения его по требованию большим объектом.

Замена одного образа другим часто бывает оправдана только в том случае, когда это происходит достаточно быстро. Если перезагрузка длится долго, то эффект теряется. Для быстрой подмены используют возможность предварительной загрузки документа в специально созданный объект класса Image.

Реальный эффект можно почувствовать только при отключении кэширования страниц на стороне клиента (браузера). Кэширование часто используют для ускорения работы со страницами Web-узла. Как правило, загрузка первой страницы – это достаточно длительный процесс. Самое главное, чтобы пользователь в этот момент был готов немного подождать. Поэтому, кроме графики, необходимой только на первой странице, ему можно передать и графику, которая на ней не отображается. Но зато при переходе к другим страницам узла она будет отображаться без задержки на передачу с сервера.